11. 성능 최적화

개요

plonky2는 고성능 영지식증명 시스템으로, 다양한 최적화 기법을 통해 빠른 증명 생성과 검증을 실현합니다. 본 문서에서는 plonky2가 사용하는 주요 성능 최적화 기법을 분석합니다.

1. 병렬 처리 전략

1.1 plonky2_maybe_rayon 모듈

plonky2는 조건부 병렬 처리를 위해 `plonky2_maybe_rayon` 모듈을 사용합니다. 이 모듈은 `parallel` 피처 플래그에 따라 Rayon 기반 병렬 처리 또는 순차 처리를 선택할 수 있게 합니다.

```
// plonky2_maybe_rayon/src/lib.rs

#[cfg(feature = "parallel")]

pub use rayon::{

self,

prelude::{

IndexedParallelIterator, ParallelDrainFull, ParallelDrainRange,

ParallelExtend, ParallelIterator,

},

};


pub trait MaybeParIter<'data> {
```

```rust
#[cfg(feature = "parallel")]
type Iter: ParallelIterator<Item = Self::Item>;


#[cfg(not(feature = "parallel"))]
type Iter: Iterator<Item = Self::Item>;


fn par_iter(&'data self) -> Self::Iter;
}
```

특징:

- 컴파일 타임에 병렬/순차 처리 선택
- 동일한 API로 두 가지 모드 지원
- 성능 측정 및 디버깅 시 유연성 제공

1.2 증명 생성에서의 병렬 처리

Wire 값 계산

증명 생성 과정에서 Wire 값을 다항식으로 변환할 때 병렬 처리를 사용합니다:

```rust
// plonky2/plonky2/src/plonk/prover.rs
use plonky2_maybe_rayon::*;


let wires_values: Vec<PolynomialValues<F>> = witness
.wire_values
.par_iter() // 병렬 반복자
```

```
.map(|column| PolynomialValues::new(column.clone()))
.collect();
```

효과:

- 각 Wire 컬럼을 독립적으로 처리
- 멀티코어 CPU 활용
- 대규모 회로에서 큰 성능 향상

Quotient Polynomial 분할

Quotient polynomial을 여러 조각으로 나눌 때도 병렬 처리를 활용합니다:

```
// plonky2/plonky2/src/plonk/prover.rs
let quotient_polys_chunks = quotient_polys
.into_par_iter() // 병렬 반복자로 변환
.flat_map(|mut quotient_poly| {
quotient_poly.trim_to_len(quotient_degree).expect("...");
quotient_poly.coeffs.chunks(chunk_size).map(|chunk| {
PolynomialCoeffs::new(chunk.to_vec())
}).collect::<Vec<_>>()
})
.collect();
```

Partial Products 계산

Permutation argument의 partial products 계산에서도 병렬 처리를 사용합니다:

```
// plonky2/plonky2/src/plonk/prover.rs
let all_quotient_chunk_products = subgroup
.par_iter() // 서브그룹의 각 원소를 병렬로 처리
.enumerate()
.map(|(i, &x)| {
// 각 점에서 quotient chunk 계산
quotient_chunk_products(/* ... */)
})
.collect();
```

1.3 Transpose 연산의 병렬화

행렬 전치(transpose) 연산은 증명 생성에서 자주 사용되며, 병렬화되어 있습니다:

```
// plonky2/plonky2/src/util/mod.rs
pub fn transpose<T: Send + Sync + Copy>(matrix: &[Vec<T>]) -> Vec<Vec<T>> {
let len = matrix[0].len();
(0..len)
.into_par_iter() // 각 열을 병렬로 처리
.map(|i| matrix.iter().map(|row| row[i]).collect())
.collect()
}
```

사용 예시:

```
// Partial products를 전치하여 다항식으로 변환
transpose(&all_partial_products_and_zs)
.into_par_iter()
.map(PolynomialValues::new)
.collect()
```

1.4 Batch 처리

대량의 데이터를 처리할 때 배치 단위로 병렬 처리합니다:

```
// plonky2/plonky2/src/plonk/prover.rs
const BATCH_SIZE: usize = 1024;

let points_batches = points.par_chunks(BATCH_SIZE);
let num_batches = points.len().div_ceil(BATCH_SIZE);

// 각 배치를 병렬로 처리
points_batches.enumerate().for_each(|(batch_idx, batch)| {
// 배치 처리 로직
});
```

2. FFT 최적화

2.1 Cache-Friendly FFT

plonky2의 FFT 구현은 캐시 효율성을 고려하여 설계되었습니다.

Bit-Reversal 최적화

FFT 알고리즘의 첫 단계인 bit-reversal은 한 번만 수행됩니다:

```
// plonky2/field/src/fft.rs
pub(crate) fn fft_classic<F: Field>(
values: &mut [F],
r: usize,
root_table: &FftRootTable<F>
) {
// 입력 배열을 bit-reversed 순서로 재배치
reverse_index_bits_in_place(values);
let n = values.len();
let lg_n = log2_strict(n);
// ... FFT 계산
}
```

Root Table 사전 계산

FFT에 필요한 unity root들을 미리 계산하여 재사용합니다:

```
// plonky2/field/src/fft.rs
pub fn fft_root_table<F: Field>(n: usize) -> FftRootTable<F> {
let lg_n = log2_strict(n);
let mut bases = Vec::with_capacity(lg_n);
```

```
let mut base = F::primitive_root_of_unity(lg_n);
bases.push(base);
// g^2^i 형태의 base들을 미리 계산
for _ in 1..lg_n {
base = base.square();
bases.push(base);
}
// 각 단계별 omega 값들을 테이블로 구성
let mut root_table = Vec::with_capacity(lg_n);
for lg_m in 1..=lg_n {
let half_m = 1 << (lg_m - 1);
let base = bases[lg_n - lg_m];
let root_row = base.powers().take(half_m.max(2)).collect();
root_table.push(root_row);
}
root_table
}
```

장점:

- 반복적인 지수 연산 제거
- 메모리 접근 패턴 개선
- 캐시 히트율 향상

2.2 SIMD 최적화

plonky2는 Packed Field를 사용하여 SIMD 연산을 지원합니다:

```
// plonky2/field/src/fft.rs
#[unroll_for_loops]
fn fft_classic_simd<P: PackedField>(
values: &mut [P::Scalar],
r: usize,
lg_n: usize,
root_table: &FftRootTable<P::Scalar>,
) {
let lg_packed_width = log2_strict(P::WIDTH);
let packed_values = P::pack_slice_mut(values);
// Packed 연산으로 여러 원소를 동시에 처리
for lg_half_m in 0..4 {
if (r..min(lg_n, lg_packed_width)).contains(&lg_half_m) {
let half_m = 1 << lg_half_m;
// omega 값을 packed 형태로 준비
let mut omega = P::default();
for (j, omega_j) in omega.as_slice_mut().iter_mut().enumerate() {
*omega_j = root_table[lg_half_m][j % half_m];
}
// SIMD 연산 수행
for k in (0..packed_n).step_by(2) {
```

```
let (u, v) = packed_values[k].interleave(packed_values[k + 1], half_m);

let  t = omega * v;

(packed_values[k], packed_values[k + 1]) = (u + t).interleave(u - t, half_m);

}

}

}

}
```

효과:

- 한 번에 여러 필드 원소 처리
- CPU의 벡터 명령어 활용
- 처리량 대폭 증가

2.3 Zero-Factor 최적화

입력의 일부가 0인 경우를 활용한 최적화:

```
// plonky2/field/src/fft.rs

pub(crate) fn  fft_classic<F: Field>(

values: &mut [F],

r: usize, // zero factor

root_table: &FftRootTable<F>

) {

reverse_index_bits_in_place(values);

// r > 0이면 입력의 마지막 1 - 1/2^r 부분이 0
```

```
// 이를 활용하여 첫 r 라운드를 최적화
if r > 0 {
let mask = !((1 << r) - 1);
for i in 0..n {
values[i] = values[i & mask];
}
}
// 나머지 FFT 계산
// ...
}
```

사용 예시:

```
// Low-degree extension 시 활용
let lde_values = coeffs.fft_with_options(Some(rate_bits), None);
```

2.4 Coset FFT 최적화

Coset 상에서의 FFT는 계수에 shift를 곱하는 방식으로 효율적으로 구현됩니다:

```
// plonky2/field/src/polynomial/mod.rs
pub fn coset_fft_with_options(
&self,
shift: F,
zero_factor: Option<usize>,
```

```
root_table: Option<&FftRootTable<F>>,
) -> PolynomialValues<F> {
// shift의 거듭제곱을 계수에 곱함
let modified_poly: Self = shift
.powers()
.zip(&self.coeffs)
.map(|(r, &c)| r * c)
.collect::<Vec<_>>()
.into();
// 일반 FFT 수행
modified_poly.fft_with_options(zero_factor, root_table)
}
```

3. 메모리 관리

3.1 Zero-Copy 기법

plonky2는 가능한 한 데이터 복사를 피하고 참조를 사용합니다:

```
// 다항식 구조체는 Vec을 직접 소유
pub struct PolynomialCoeffs<F: Field> {
pub coeffs: Vec<F>,
}
```

```rust
pub struct PolynomialValues<F: Field> {
pub values: Vec<F>,
}

// FFT는 소유권을 가져와서 in-place 변환
pub fn fft<F: Field>(poly: PolynomialCoeffs<F>) -> PolynomialValues<F> {
let PolynomialCoeffs { coeffs: mut buffer } = poly;
fft_dispatch(&mut buffer, None, None);
PolynomialValues::new(buffer) // 버퍼 재사용
}
```

3.2 In-Place 연산

많은 연산이 in-place로 수행되어 메모리 할당을 최소화합니다:

```rust
// Bit-reversal을 in-place로 수행
reverse_index_bits_in_place(values);

// FFT도 in-place로 수행
fft_classic(values, r, root_table);
```

3.3 Polynomial 재사용

다항식 객체를 재사용하여 할당 오버헤드를 줄입니다:

```rust
// 패딩을 통한 재사용
pub fn pad(&mut self, new_len: usize) -> Result<()> {
ensure!(new_len >= self.len(), "...");
self.coeffs.resize(new_len, F::ZERO); // 기존 버퍼 확장
Ok(())
}

// LDE는 패딩 후 FFT
pub fn lde(&self, rate_bits: usize) -> Self {
self.padded(self.len() << rate_bits)
}
```

3.4 Batch 연산

여러 다항식을 한 번에 처리하여 메모리 접근 패턴을 개선합니다:

```rust
pub fn lde_multiple(polys: Vec<&Self>, rate_bits: usize) -> Vec<Self> {
polys.into_iter().map(|p| p.lde(rate_bits)).collect()
}
```

4. 성능 벤치마크

4.1 FFT 벤치마크

plonky2는 다양한 크기의 FFT에 대한 벤치마크를 제공합니다:

```
// plonky2/plonky2/benches/ffts.rs
pub(crate) fn bench_ffts<F: Field>(c: &mut Criterion) {
let mut group = c.benchmark_group(format!("fft<{}>", type_name::<F>()));

for size_log in [13, 14, 15, 16] {
let size = 1 << size_log;
group.bench_with_input(BenchmarkId::from_parameter(size), &size, |b, _| {
let coeffs = PolynomialCoeffs::new(F::rand_vec(size));
b.iter(|| coeffs.clone().fft_with_options(None, None));
});
}
}
```

일반적인 성능:

- 2^13 (8,192) 원소: ~0.5ms
- 2^14 (16,384) 원소: ~1ms
- 2^15 (32,768) 원소: ~2ms
- 2^16 (65,536) 원소: ~4ms

4.2 LDE 벤치마크

Low-degree extension의 성능 측정:

```
pub(crate) fn bench_ldes<F: Field>(c: &mut Criterion) {
```

```
const RATE_BITS: usize = 3; // 8배 확장

let mut group = c.benchmark_group(format!("lde<{}>", type_name::<F>()));

for size_log in [13, 14, 15, 16] {
let orig_size = 1 << (size_log - RATE_BITS);
let lde_size = 1 << size_log;

group.bench_with_input(BenchmarkId::from_parameter(lde_size), &lde_size, |b, _|
let coeffs = PolynomialCoeffs::new(F::rand_vec(orig_size));
b.iter(|| {
let padded_coeffs = coeffs.lde(RATE_BITS);
padded_coeffs.fft_with_options(Some(RATE_BITS), None)
});
});
}
}
```

4.3 Transpose 벤치마크

행렬 전치 연산의 성능:

```
// plonky2/plonky2/benches/transpose.rs
fn criterion_benchmark(c: &mut Criterion) {
```

```
type F = GoldilocksField;

const WIDTH: usize = 1 << 16; // 65,536


let mut group = c.benchmark_group("transpose");


// 다양한 높이의 행렬 테스트

for height in [5, 50, 100, 150] {

group.bench_with_input(BenchmarkId::from_parameter(height), &height, |b, _| {

let matrix = (0..height).map(|_| F::rand_vec(WIDTH)).collect::<Vec<_>>();

b.iter(|| transpose(&matrix));

});

}

}
```

성능 특성:

- 병렬 처리로 높이가 증가해도 선형적 성능 유지
- 캐시 효율성이 중요한 연산

5. 최적화 전략 요약

5.1 병렬 처리

- Rayon을 통한 데이터 병렬성 활용
- 독립적인 연산을 병렬로 수행

- 멀티코어 CPU 효율적 활용

5.2 FFT 최적화

- Cache-friendly 알고리즘
- SIMD 명령어 활용
- Root table 사전 계산
- Zero-factor 최적화

5.3 메모리 최적화

- Zero-copy 설계
- In-place 연산
- 버퍼 재사용
- Batch 처리

5.4 알고리즘 최적화

- Bit-reversal 최소화
- Coset FFT 효율적 구현
- Transpose 병렬화

6. 실전 활용 팁

6.1 병렬 처리 활성화

Cargo.toml에서 parallel 피처를 활성화합니다:

```
[dependencies]
```

```
plonky2 = { version = "0.2", features = ["parallel"] }
```

6.2 Root Table 재사용

동일한 크기의 FFT를 여러 번 수행할 때는 root table을 재사용합니다:

```
let root_table = fft_root_table::<F>(n);

for poly in polynomials {
let values = poly.fft_with_options(None, Some(&root_table));
// ...
}
```

6.3 Batch 연산 활용

여러 다항식을 처리할 때는 batch 함수를 사용합니다:

```
let lde_polys = PolynomialCoeffs::lde_multiple(polys, rate_bits);
```

6.4 메모리 프로파일링

대규모 회로의 경우 메모리 사용량을 모니터링합니다:

```
// 회로 크기 확인
println!("Degree: {}", common_data.degree());
```

```
println!("Quotient degree factor: {}", common_data.quotient_degree_factor);
```

관련 문서

- 01. 아키텍처: 전체 구조 이해
- 03. 증명 생성: 증명 생성 과정에서의 최적화
- 05. FRI 프로토콜: FRI에서의 FFT 활용
- 06. Goldilocks Field: 필드 연산 최적화

문서 네비게이션

- **이전**: 10. 재귀 증명 - 재귀 증명 구성
- **다음**: 00. 인덱스 - 문서 시리즈 개요로 돌아가기
- **관련 문서**:
- 01. 아키텍처 개요 - 전체 구조와 최적화 요소
- 03. 증명 생성 프로세스 - 증명 생성 최적화
- 05. FRI 프로토콜 - FRI에서의 FFT 활용
- 06. Goldilocks Field - 필드 연산 최적화